SR UNIVERSITY

School of Computer Science and Artificial Intelligence

Lab Assignment # 13.2

Program	**: B. Tech (CSE)**
Specialization	**: -**
Course Title	**: AI Assisted Coding**
Course Code	**: 23CS002PC304**
Semester	**: II**
Academic Session	**: 2025-2026**
Name of Student	**: K.Pavan**
Enrollment No.	**: 2403A51L19**
Batch No.	**: 51**
Date	**: 10/03/26**

Submission Starts here

Screenshots:

Task Description -1 (Refactoring – Eliminating Repetitive Logic)

Prompt:
Refactor the following Python program to eliminate repetitive logic.

Requirements:
1. **Identify repeated calculations in the code.**
2. **Encapsulate the repeated logic into a reusable function.**
3. **The function should compute both area and perimeter of a rectangle.**
4. **Ensure the output remains exactly the same as the original program.**
5. **Add proper Python docstrings to the function explaining parameters and return values.**
6. **Keep the code clean, readable, and maintainable.**

Here is the original code:

```
length = 8 width = 4
print("Area:", length * width)
print("Perimeter:", 2 * (length + width))
```

length = 12
width = 6 print("Area:", length * width)
print("Perimeter:", 2 * (length + width))
Output:

```python
def rectangle_properties(length, width):
    """Calculate the area and perimeter of a rectangle.

    Parameters
    ----------
    length : float or int
        The length of the rectangle.
    width : float or int
        The width of the rectangle.

    Returns
    -------
    tuple
        A tuple of two values: (area, perimeter).
    """
    area = length * width
    perimeter = 2 * (length + width)
    return area, perimeter


length = 8
width = 4
area, perimeter = rectangle_properties(length, width)
print("Area:", area)
print("Perimeter:", perimeter)

length = 12
width = 6
area, perimeter = rectangle_properties(length, width)
print("Area:", area)
print("Perimeter:", perimeter)
```

Task Description -2 (Refactoring – Modularizing Inline Calculations)

Prompt:

Refactor the following Python code by extracting repeated inline calculations into a reusable function.

Requirements:
1. Detect repeated calculation patterns.
2. Create a reusable function to compute the final amount after applying discount.
3. Add a clear docstring explaining the function purpose, parameters, and return value.
4. Ensure the final output remains the same as the original program.
5. Improve readability and modularity.

Original Code:

amount = 1000 discount = amount
* 0.10 final_amount = amount -
discount
print("Final Amount:", final_amount)

amount = 2000 discount = amount
* 0.10 final_amount = amount -
discount

print("Final Amount:", final_amount)

Output:

```python
def calculate_final_amount(amount, discount_rate=0.10):
    """
    Calculate the final amount after applying a discount.

    Parameters:
        amount (float or int): The original amount before discount.
        discount_rate (float): The discount rate to apply (e.g., 0.10 for 10%).

    Returns:
        float: The final amount after subtracting the discount.
    """
    discount = amount * discount_rate
    return amount - discount


amount = 1000
final_amount = calculate_final_amount(amount)
print("Final Amount:", final_amount)

amount = 2000
final_amount = calculate_final_amount(amount)
print("Final Amount:", final_amount)
```

Task Description -3 (Refactoring – Improving Loop and Conditional Performance)

Prompt:

Optimize the following Python program to improve performance.

Requirements:
1. Identify inefficient nested loops and repeated conditional checks.
2. Replace the inefficient lookup logic with a more efficient data structure such as a set.
3. Maintain the same program output.
4. Provide both the original and optimized versions.
5. Measure and compare execution time of both versions using the time module.
6. Add comments explaining the optimization.

Original Code:

```
students = ["Anil", "Bhavya", "Charan", "Divya"] check_list
= ["Charan", "Esha", "Bhavya"]

for name in check_list:
    exists = False     for
student in students:
if name == student:
            exists = True
     if exists:
```

```
        print(name, "found")
else:
        print(name, "not found")
```

Output:

```python
import time
# Original code (baseline version)
students = ["Anil", "Bhavya", "Charan", "Divya"]
check_list = ["Charan", "Esha", "Bhavya"]

def original_check(students_list, check_list_local):
    """
    Original implementation using nested loops.
    Time complexity: O(len(students_list) * len(check_list_local))
    """
    for name in check_list_local:
        exists = False
        for student in students_list:
            if name == student:
                exists = True
        if exists:
            print(name, "found")
        else:
            print(name, "not found")

# Optimized code using set lookups
# Optimization:
# - Convert the students list to a set once.
# - Membership check 'name in students_set' is O(1) on average.
# - This removes the inner loop and repeated conditional checks.
students_set = set[str](students)  # Precompute for faster membership tests


def optimized_check(students_set_local, check_list_local):
    """
    Optimized implementation using a set for O(1) lookups.
    Time complexity: O(len(students_set_local) + len(check_list_local))
    """
    for name in check_list_local:
        if name in students_set_local:  # Direct, efficient membership check
            print(name, "found")
        else:
            print(name, "not found")


if __name__ == "__main__":
    print("Original Version Output:")
    original_check(students, check_list)

    print("\nOptimized Version Output:")
    optimized_check(students_set, check_list)

    # Timing comparison
    # To see a measurable difference, run each version many times.
    ITERATIONS = 10000

    start = time.time()
    for _ in range(ITERATIONS):
        original_check(students, check_list)
    original_time = time.time() - start

    start = time.time()
    for _ in range(ITERATIONS):
        optimized_check(students_set, check_list)
    optimized_time = time.time() - start

    print("\n--- Timing Comparison ---")
    print("Original version time :", original_time, "seconds")
    print("Optimized version time:", optimized_time, "seconds")
```

Task Description -4 (Refactoring – Replacing Hardcoded Values with Constants)

Prompt:

Refactor the following Python code to replace hardcoded numeric values with named constants.
Requirements:
1. Identify all hardcoded numeric values.

2. Create descriptive constants at the top of the program.
3. Use constant names like RATE and TIME instead of raw values.
4. Ensure the program output remains unchanged.
5. Improve code readability and maintainability.
6. Add comments explaining the constants.

Original Code:

print("Simple Interest:", (1000 * 2 * 5) / 100)
print("Simple Interest:", (2000 * 2 * 5) / 100) Output:

```python
"""
Program to calculate simple interest for two different principal amounts
using descriptive constants instead of hardcoded numeric values.
"""

# Principal amounts (in currency units)
PRINCIPAL_1 = 1000
PRINCIPAL_2 = 2000

# Simple interest rate (in percent)
RATE = 2

# Time period (in years)
TIME = 5

# Simple Interest = (Principal * Rate * Time) / 100
print("Simple Interest:", (PRINCIPAL_1 * RATE * TIME) / 100)
print("Simple Interest:", (PRINCIPAL_2 * RATE * TIME) / 100)
Ctrl+L to chat, Ctrl+K to generate
```

Task Description -5 (Re factoring – Enhancing Variable Naming and Code Clarity)

Prompt:

Improve the readability of the following Python program.

Requirements:
1. Replace unclear variable names with meaningful descriptive names.
2. Add inline comments explaining the logic.
3. Ensure the program output remains exactly the same.
4. Follow Python naming conventions.
5. Make the code easier to understand for beginners.

Original Code:

x = 50 y
= 30 z =
x + y
print(z)

Output:

```
first_number = 50
second_number = 30

# Add the two numbers together
sum_of_numbers = first_number + second_number

# Print the result (same output as before)
print(sum_of_numbers)
Ctrl+L to chat, Ctrl+K to generate
```